**REGULAR ARTICLE**

**Systems Engineering Journal**

# Understanding Software Engineering Organizations Through Systems Thinking

**Juliana Alves**[1] | **Henrique Alves**[2, 3] | **André Costa Batista**[1, 4]

[1]Systems Engineer, Steiner Institute, Cherenkov Telescope Array Observatory (CTAO) - São Paulo, SP, Brazil

[2]Product Manager, Be Finance, W1 Inc. - Alameda Oscar Niemeyer, 132, 34006-049, Nova Lima, MG, Brazil

[3]Graduate Program in Electrical Engineering - Universidade Federal de Minas Gerais - Av. Antônio Carlos 6627, 31270-901, Belo Horizonte, MG, Brazil

[4]Department of Electrical Engineering - Universidade Federal de Minas Gerais - Av. Antônio Carlos 6627, 31270-901, Belo Horizonte, MG, Brazil

**Correspondence**
André Costa Batista, Department of Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antônio Carlos 6627, 31270-901, Belo Horizonte, MG, Brazil
Email: andre-costa@ufmg.br

**Present address**
Department of Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antônio Carlos 6627, 31270-901, Belo Horizonte, MG, Brazil

**Funding information**
Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES), Grant/Award Number: 001; Pró-Reitoria de Pesquisa da Universidade Federal de Minas Gerais (PRPq/UFMG), under Call 10/2024.

Software engineering organizations operate as complex adaptive systems where technical decisions, organizational structures, and market forces interact through feedback loops to produce emergent outcomes. Despite this inherent systemic nature, existing research predominantly examines isolated components rather than their dynamic interdependencies, limiting understanding of why common strategies succeed or fail. This research develops an agent-based model of software engineering organizations as complex adaptive systems, suggesting critical resource thresholds determining survival. Through 240 simulations, we identify binary survival gates: minimum runway sufficient to bridge profitability gaps (200 months in the baseline parameterization), dual commercial capacity thresholds (≥6 staff, ≥30% composition), and quality-velocity regimes. The absolute cutoff values are model-dependent and should not be interpreted literally; the central finding is the evidence for the existence of threshold effects and their implications for strategy. **Keywords** — Systems Thinking, Software Engineering Organizations, Agent-Based Modeling, Complex Adaptive Systems

**Abbreviations:** ABM, Agent-Based Modeling; TD, Technical Debt; SD, System Dynamics; IT, Information Technology; HR, Human Resources; PMF, Product-Market Fit

## 1 | INTRODUCTION

Software engineering has evolved from a primarily technical discipline to a complex socio-technical endeavor encompassing organizational dynamics, human factors, and strategic decision-making (Jr., 1995; Sommerville, 2016). The success or failure of software ventures increasingly depends on the interplay between technical decisions, organizational structure, and market forces, yet traditional approaches often focus on isolated components rather than their interactions (Boehm and Turner, 2003).

Systems thinking provides a framework for understanding such complex adaptive systems, emphasizing circular causality, feedback mechanisms, and emergent properties (Senge, 1990; Sterman, 2000; Meadows, 2008). In software engineering, it explains counterintuitive phenomena: how increasing team size can decrease productivity (Brooks's Law), how focusing solely on speed can paradoxically slow development through technical debt accumulation, or how optimizing individual team performance may undermine overall effectiveness (Repenning, 2001; Miller and Page, 2007). Despite this, existing research predominantly examines either technical or organizational factors in isolation, rarely modeling their co-evolution (Kruchten et al., 2012). Empirical studies employ linear statistical models that fail to capture feedback effects and non-linear relationships (Madachy, 2008). Agent-based modeling (ABM) is a potential approach to bridge this gap (North and Macal, 2007; Wilensky and Rand, 2015), yet its application to software engineering organizations remains nascent.

This article addresses these gaps with three contributions: (1) a comprehensive ABM representing key stakeholders (engineers, sales, marketing, HR, management) as autonomous agents whose interactions generate emergent organizational outcomes; (2) systematic exploration through three computational experiments examining team composition, quality-speed tradeoffs, and financial constraints, revealing threshold effects and non-linear dynamics; and (3) a methodological blueprint for applying systems thinking and ABM to software engineering research. The remainder is organized as follows: Section 2 reviews related work, Section 3 describes the methodology, Section 4 reports experimental results, and Section 5 concludes with implications and limitations.

## 2 | RELATED WORK

Our work builds upon three research streams: systems thinking in software engineering, agent-based modeling of organizations, and empirical studies of team performance.

**Systems thinking and feedback in software engineering.** Lehman (1996) and Lehman and Ramil (1999) established that software evolution constitutes a multi-loop feedback system where self-stabilizing mechanisms constrain process behavior, requiring explicit modeling of global feedback loops rather than isolated technical innovations. Franco et al. (2023) operationalized these mechanisms for technical debt management, formulating interacting feedback loops explaining how shortcuts degrade maintenance productivity. Lee and Miller (2004) demonstrated feedback loops between workload, exhaustion, and productivity in multi-project environments, while Chernoguz (2011) used System Dynamics to investigate Brooks's Law boundaries, showing that productivity decreases linearly with team size due to coordination burdens but that mentoring investment can mitigate schedule slippage. These works establish the theoretical importance of feedback mechanisms but remain limited to particular contexts without integrating technical, organizational, and market dimensions.

**Agent-based modeling of organizations.** Crowder et al. (2012) developed a seminal ABM for Integrated Product Teams with empirically-grounded agent behaviors (competency, motivation, trust, availability), demonstrating that competency dominates work quality while availability determines completion time. Lapp et al. (2019) introduced cog-

nitive style into team problem-solving simulations, revealing that optimal strategies depend on problem characteristics and team composition. Meluso et al. (2019, 2022) linked organizational miscommunication to system performance through the CESIUM framework, showing convergence time scales with network size. Systematic reviews (Sandria-Flores et al., 2024; Negahban and Yilmaz, 2014) identify critical limitations: prevalence of abstract models lacking empirical grounding and need for robust verification. Zali et al. (2014) found computational modeling underutilized for strategic decision support in venture contexts. While these approaches capture team dynamics effectively, they predominantly focus on engineering design teams, not the full socio-technical complexity of software organizations including market forces and financial constraints.

**Organizational dynamics and empirical studies.** Blackburn et al. (2006) validated Brooks's Law empirically across 117 projects, showing complexity affects productivity indirectly through team size. Hindiyeh et al. (2023) reviewed 53 articles on team dynamics, finding the field heavily skewed toward IT with limited methodological rigor. Petkov et al. (2008) reconceptualized organizations as information processing systems, explaining hierarchies as bandwidth management mechanisms. Surveys by Fan and Yen (2004), Wynn and Clarkson (2018), and Costa et al. (2020) reveal persistent gaps between theoretical formulations and industrial implementation.

Our work addresses these gaps by developing an integrated ABM that unifies technical, organizational, and market dimensions within a feedback-driven framework, capturing the full venture lifecycle including financial sustainability, strategic pivots, and resource allocation under uncertainty.

## 3 | METHODOLOGY

Our methodological approach integrates agent-based modeling with systems thinking to represent software engineering organizations as complex adaptive systems. This section describes the model architecture, agent types and behaviors, feedback mechanisms, and experimental design.

### 3.1 | Model Architecture and Design Philosophy

We developed an agent-based model implemented in Python using the Mesa framework (Wilensky and Rand, 2015), enabling discrete-event simulation of organizational dynamics over time. Each simulation step represents one month of organizational operation, chosen to balance computational efficiency with meaningful strategic decision intervals. The model architecture follows a modular design separating three interconnected subsystems: (1) the *Technical Subsystem* governing product evolution through feature development, code quality, technical debt, and defects; (2) the *Market Subsystem* capturing customer acquisition, retention, brand awareness, and competitive positioning; and (3) the *Organizational Subsystem* representing team dynamics, alignment, conflicts, and resource allocation.

Unlike traditional equation-based models that impose top-down behavioral assumptions, our ABM approach allows organizational outcomes to emerge from bottom-up interactions among heterogeneous agents operating under local decision rules (North and Macal, 2007). This generative methodology enables exploration of non-obvious dynamics, path dependencies, and threshold effects that characterize real software ventures but resist analytical tractability.

### 3.2 | Agent Types and Capabilities

The model represents five distinct stakeholder types, each with specialized capabilities that contribute to organizational performance. Table 1 summarizes agent types and their primary attributes.

**TABLE 1** Agent types, capabilities, and organizational roles

| Agent Type | Capabilities | Primary Functions |
|---|---|---|
| Engineer | Development Capacity (30-100), Explanation Capacity (30-100) | Feature implementation, bug fixing, refactoring, technical debt management |
| Sales | Selling Capacity (30-100), Understanding Capacity (30-100) | Customer acquisition, market feedback, revenue generation |
| Marketing | Improvement Ideas Capacity (30-100), Publicize Capacity (30-100) | Brand building, lead generation, market insight |
| Human Resources | Coordination Capacity (30-100), Talent Development (30-100), Conflict Resolution (30-100), Hiring Quality (30-100) | Team alignment, turnover reduction, skill development, recruitment |
| Management | Management Capacity (30-100), Strategic Vision (30-100), Crisis Management (30-100) | Strategic focus, resource allocation, crisis response |

Each agent's capabilities are initialized randomly within [30, 100], representing organizational talent heterogeneity. The lower bound reflects hiring thresholds; initial assignments follow uniform distributions, abstracting from extreme talent heterogeneity (e.g., "10x engineers") — Section 5 discusses implications. Capabilities evolve through learning mechanisms modulated by product state, organizational alignment, and strategic focus.

## 3.3 | Product State Variables

The software product evolves through ten state variables representing technical, market, and organizational dimensions (Table 2).

**TABLE 2** Product state variables and their dynamics

| Variable | Range | Description |
|---|---|---|
| Feature Completeness | $[0, \infty)$ | Accumulated functionality scope |
| Code Quality | $[0, 100]$ | Internal code maintainability |
| Technical Debt | $[0, \infty)$ | Accumulated design shortcuts |
| Bug Count | $[0, \infty)$ | Known defects inventory |
| Market Fit | $[0, 100]$ | Product-market need alignment |
| Market Share | $[0, 100]$ | Percentage of market captured |
| Brand Awareness | $[0, 100]$ | Market recognition level |
| Lead Quality | $[0, 100]$ | Sales pipeline qualification |
| Organizational Alignment | $[0, 100]$ | Strategic coherence across teams |
| Organizational Conflict | $[0, 100]$ | Internal friction level |

These variables evolve according to differential equations driven by agent actions and modulated by feedback mechanisms (detailed in Supplementary Material). Feature completeness increases with development effort but is penalized by technical debt drag, implementing the widely observed phenomenon that shortcuts accelerate initial development but compound into long-term velocity loss (Franco et al., 2023). Technical debt accumulates proportionally to development work modulated by a quality focus factor $\tau = \min(C_{exp}/100, 0.8)$, where teams with low explanation capacity generate debt at rate $0.7\times$ effort, while high-capacity teams generate debt at $0.2\times$ effort. Both technical debt and bugs impose non-linear drag on development velocity:

$$V_{effective} = V_{nominal} \cdot \frac{1}{1 + (D/50)^2} \cdot \frac{1}{1 + (B/20)^{1.5}} \tag{1}$$

ensuring that high debt ($D > 100$) or critical bug counts ($B > 30$) severely impair productivity. Market share grows through sales effort modulated by product quality and brand awareness, exhibiting saturation dynamics. Organizational alignment increases through management and HR coordination but decays naturally, requiring continuous maintenance.

## 3.4 | Feedback Loop Structure

The model implements eight primary feedback loops (four reinforcing, four balancing) that govern system behavior. Figure 1 presents the causal loop diagram. Detailed mathematical formulations are provided in the Supplementary Material.

**Reinforcing loops:** R1 (Short-Term Focus) drives growth through feature-market-revenue coupling. R2 (Bug Accumulation) captures catastrophic defect cascades when bug count exceeds resolution capacity. R3 (Brand-Sales Synergy) creates compounding returns to coordinated commercial activity. R4 (Alignment-Performance Spiral) couples organizational alignment with performance outcomes.

**Balancing loops:** B1 (Technical Debt Brake) implements the quality-velocity tradeoff (Lehman, 1996; Franco et al., 2023) — development shortcuts generate technical debt imposing drag on velocity. B2 (Market Saturation) implements diminishing returns through a $(1 - M)^2$ saturation function. B3 (Quality Investment Tradeoff) governs allocation between feature development and quality improvement. B4 (Conflict Dynamics) represents conflict generation and resolution through HR mechanisms.

## 3.5 | Strategic Decision Mechanisms

Management agents assess organizational state periodically and execute strategic interventions based on heuristics implementing bounded rationality (Petkov et al., 2008): technical debt crisis response (shift to quality-focused mode when debt exceeds 150), bug crisis response (divert engineering to bug resolution when count exceeds 30), market fit recovery (increase marketing/sales when fit below 40), growth investment (hire when runway exceeds 50 months and capacity is high), pivot execution (reset product direction retaining team experience), and emergency layoffs (reduce headcount when runway below 5 months).

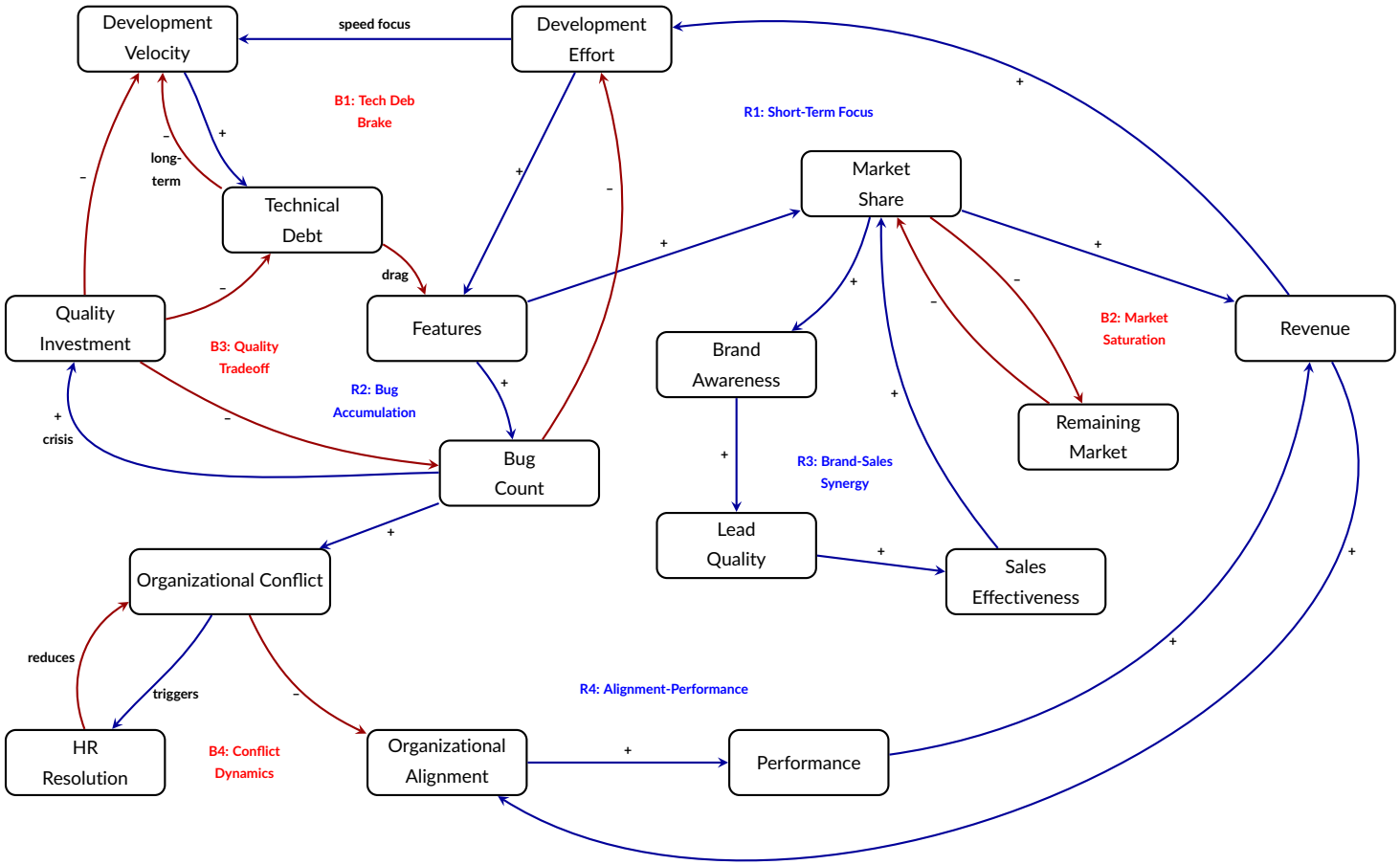


**FIGURE 1** Eight primary feedback loops governing organizational dynamics. Reinforcing loops (R1-R4, blue) amplify growth or decline; balancing loops (B1-B4, red) provide constraints and self-regulation. Loop R1 drives growth through feature-market-revenue coupling. B1 implements technical debt drag on velocity. R2 captures bug cascades when defects exceed resolution capacity. B2 prevents unrealistic market saturation. R3 creates brand-sales synergy. B3 governs quality-velocity tradeoffs. R4 couples alignment with performance. B4 represents conflict generation and resolution dynamics.

## 3.6 | Financial Dynamics and Resource Constraints

The model incorporates explicit financial constraints through cash runway dynamics (Zali et al., 2014). Revenue depends multiplicatively on market share and product quality (penalized by bugs and debt), while burn rate scales linearly with headcount plus fixed overhead (Madachy, 2008). Three terminal failure conditions are implemented: financial exhaustion ($R \leq 0$), technical death spiral (debt $> 200$), and product collapse (bugs $> 100$). Funding rounds occur when runway falls below 20 months, with injection proportional to product quality and market traction.

## 3.7 | Team Evolution and Turnover

Agent capabilities evolve dynamically: learning increases with organizational alignment and HR talent development, while degradation accelerates under stress (high bugs, conflict). Turnover occurs probabilistically with base rate 2%/month, modulated by HR effectiveness and conflict level. Departed agents are replaced through hiring where recruit capabilities depend on HR hiring quality. The complete simulation algorithm is provided in the Supplementary Material.

## 3.8 | Experimental Design

We conducted three systematic experiments to explore organizational dynamics under varying conditions (Table 3). Each experiment manipulates specific parameters or initial conditions while holding others constant, enabling isolation of causal mechanisms.

**TABLE 3** Experimental scenarios and research questions

| Experiment | Manipulation | Research Question |
|---|---|---|
| E1: Unbalanced Teams | Engineer-heavy vs. sales-heavy vs. balanced | Can technical excellence compensate for weak commercialization? |
| E2: Quality vs. Speed | Vary $\alpha_{quality}$ strategy | When does quality investment pay off? |
| E3: Initial Capital | Vary starting cash runway | Does extended runway improve outcomes or enable waste? |

Each experiment runs 20 replications with different random seeds to account for stochastic variation. Simulations extend for 200 months (16+ years) or until venture termination. We collect time-series data on all state variables and analyze both trajectory patterns and terminal outcomes.

# 4 | RESULTS

We present results from three computational experiments designed to explore organizational dynamics under varying configurations. Each experiment runs 20 replications across parameter values to account for stochastic variation, with simulations extending 200 months or until venture termination. This section reports findings for each experiment, identifying non-linear relationships, threshold effects, and counterintuitive dynamics that emerge from feedback loop

interactions. All model code and experimental configurations are available in the project repository for reproducibility: `www.github.com/andre-batista/useotst`

## 4.1 | Experiment 1: Unbalanced Team Configurations

Experiment 1 tests extreme team compositions spanning tech-heavy (80% engineers), sales-heavy (55% sales), marketing-heavy (44% marketing), balanced, and minimal (12 total) configurations (Table 4). Each runs 20 replications across headcounts from 12 to 27.

**TABLE 4** Experiment 1: Unbalanced team configurations (20 replications each)

| Configuration | Eng | Sales | Mkt | HR | Mgmt | Total | Strategy |
|---|---|---|---|---|---|---|---|
| Tech-Heavy | 20 | 2 | 1 | 1 | 1 | 25 | Product excellence |
| Sales-Heavy | 5 | 15 | 5 | 1 | 1 | 27 | Market conquest |
| Balanced | 10 | 10 | 5 | 1 | 1 | 27 | Equilibrium |
| Marketing-Heavy | 8 | 5 | 12 | 1 | 1 | 27 | Brand building |
| Minimal | 5 | 3 | 2 | 1 | 1 | 12 | Lean startup |

Results indicate significant bifurcation (Table 5): Tech-Heavy achieves 0% survival (failing at 14.75 months), Minimal achieves 35% survival (failing at 19.85 months), while commercial-capable configurations (Sales-Heavy, Balanced, Marketing-Heavy) achieve 100% survival.

**TABLE 5** Experiment 1: Aggregate performance by configuration

| Configuration | Survival | Time (mo) | Mkt Share | Revenue | Tech Debt | Bugs |
|---|---|---|---|---|---|---|
| Tech-Heavy | 0% | 14.75 ± 1.07 | 0.00 | 3.43 ± 0.20 | 21.23 ± 0.94 | 8.54 ± 0.49 |
| Minimal | 35% | 19.85 ± 2.97 | 1.63 ± 2.30 | 29.04 ± 34.47 | 24.21 ± 2.62 | 6.86 ± 5.29 |
| Marketing-Heavy | 100% | — | 7.52 ± 0.88 | 120.30 ± 18.11 | 21.01 ± 0.92 | 0.00 |
| Balanced | 100% | — | 12.80 ± 1.10 | 234.20 ± 26.10 | 20.86 ± 0.57 | 0.00 |
| Sales-Heavy | 100% | — | 17.70 ± 1.58 | 341.41 ± 37.96 | 21.22 ± 0.86 | 0.00 |

A dual-threshold structure governs viability. First, commercial proportion threshold: Tech-Heavy (8% commercial) fails universally from revenue incapacity despite 20 engineers. Second, absolute capacity threshold: Minimal (5 commercial staff, 42% proportion) experiences 65% failure despite adequate proportion — insufficient absolute headcount triggers quality-driven collapse (debt=24.21, bugs=6.86). Survival requires both a commercial proportion greater than 30% and an absolute commercial staff of at least 6.

Among survivors, Sales-Heavy dominates with 17.70% market share and $341.41 revenue — 46% higher than Balanced despite identical headcount (Figure 2). Marketing-Heavy underperforms substantially (7.52% share, $120.30 revenue), revealing that conversion capacity dominates demand generation.

Survivors converge to uniform quality (features 116.9–118.0, debt ≈21.0, bugs 0.0), eliminating technical differentiation. Tech-Heavy achieves higher code quality (70.6) than survivors (∼60.0) yet fails catastrophically, demonstrating that local code cleanliness proves irrelevant without commercial viability.

Time-series analysis (Figure 3) reveals distinct failure mechanisms. Tech-Heavy experiences financial collapse

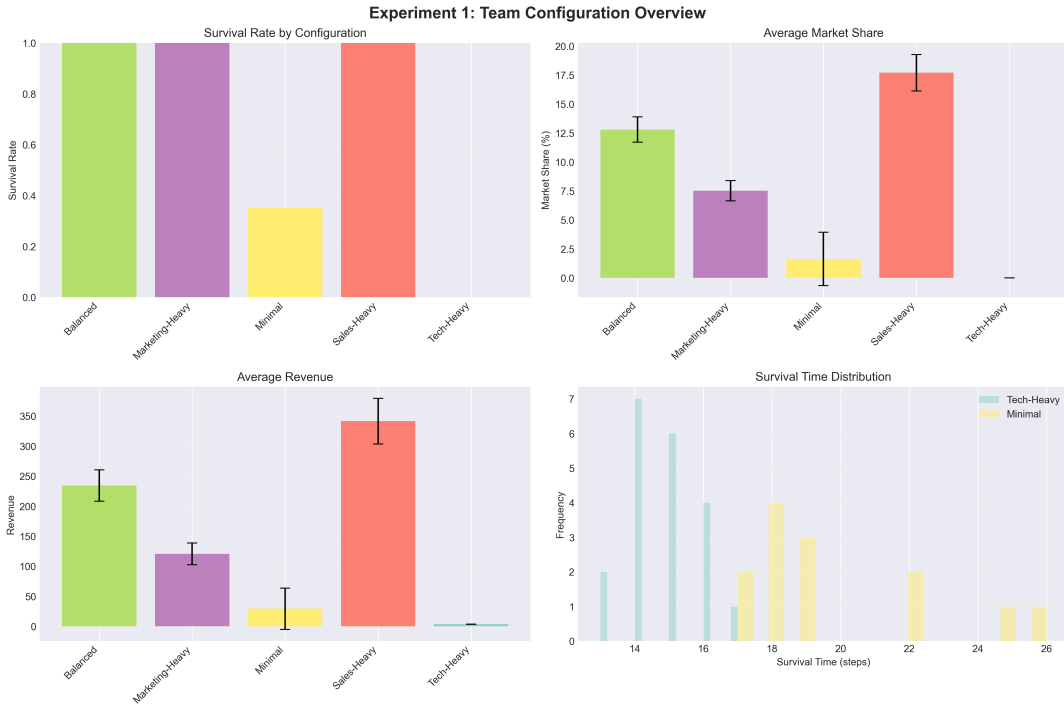


**FIGURE 2** Experiment 1: Team configuration performance overview. Top-left: Survival bifurcates dramatically. Top-right: Market share varies 10.9× (0-17.70%), with Sales-Heavy leading survivors. Bottom-left: Revenue exhibits 11.8× variation, stratified by commercial capacity. Bottom-right: Survival time distributions show Tech-Heavy modal failure at 14-15 months.

at 14.75 months from pure commercial incapacity. Minimal demonstrates delayed quality-driven collapse at 19.85 months: 5 engineers prove insufficient under resource pressure, triggering debt accumulation (Loop B1) and bug cascades (Loop R2).

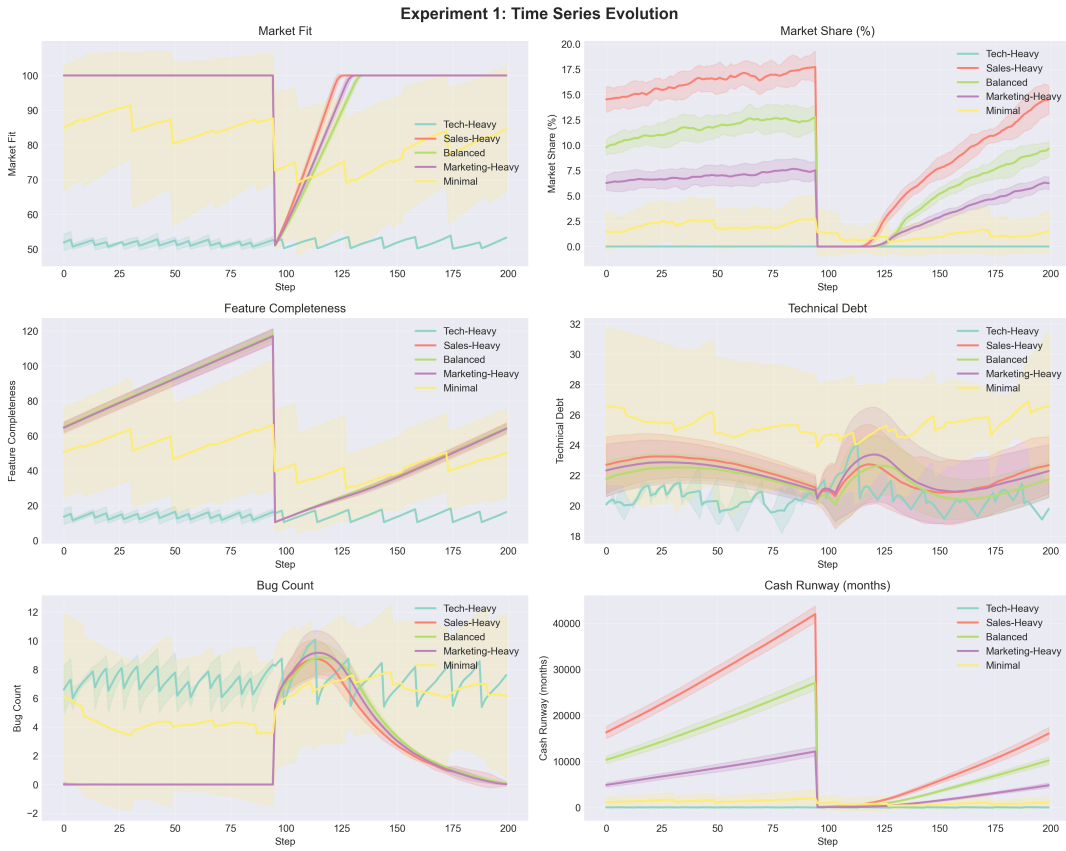


**FIGURE 3** Experiment 1: Time-series evolution reveals dual failure modes. Top-left: Market fit bifurcates sharply. Top-right: Market share shows exponential divergence—Sales. Middle-left: Feature completeness stratifies dramatically. Middle-right: Technical debt exhibits transient spikes for survivors. Bottom-left: Bug count spikes catastrophically for failures. Bottom-right: Cash runway shows exponential accumulation for survivors.

Strategic implications indicate minimum viable capacity: ≥8 engineers, ≥6 commercial staff, ≥15 total headcount. Below these thresholds, organizations entered stochastic failure zones. Sales capacity provides greater leverage than marketing (135% market share advantage, 184% revenue advantage). Among survivors, engineering headcount variation produces indistinguishable quality outcomes, suggesting an engineering sufficiency threshold beyond which additional capacity should redirect to commercial functions. Organizational alignment and conflict exhibit minimal differentiation across configurations — survivors maintain 100.0 alignment and zero conflict, confirming that financial constraints and quality degradation dominate survival outcomes under current parameterization.

## 4.2 | Experiment 2: Quality vs. Speed Development Strategies

Experiment 2 tests quality-velocity tradeoffs (Lehman, 1996; Franco et al., 2023) by comparing five strategies: Technical Debt Spiral (fastest, 0.025 features/step), Move Fast Break Things, Balanced Pragmatic, Quality First, and Extreme Quality (slowest, 0.005 features/step). All configurations maintain identical teams (10 engineers, 5 sales, 3 marketing, 1 HR, 1 management) and capital (50 months), isolating strategy effects.

Results reveal survival bifurcation: Technical Debt Spiral achieves 0% survival (collapsing at $27.75 \pm 5.82$ months with debt=68.14, bugs=37.42), while four other strategies achieve 100% survival (Table 6). This demonstrates Loops B1 and R2 engaging destructively when debt generation exceeds refactoring capacity.

Quality First outperforms Move Fast Break Things despite a 2.4× slower initial velocity. Over 200 months, Quality First delivers 139.89 features versus 57.91, generates $142.20 in revenue compared to $41.07 (246% higher), and achieves a sustainability index of 0.998 versus 0.269. This advantage is explained by debt accumulation: Move Fast accrues a technical debt of 110.20, imposing a 71% revenue penalty through Loop B1, whereas Quality First maintains zero debt, enabling sustained development productivity (Figure 4).

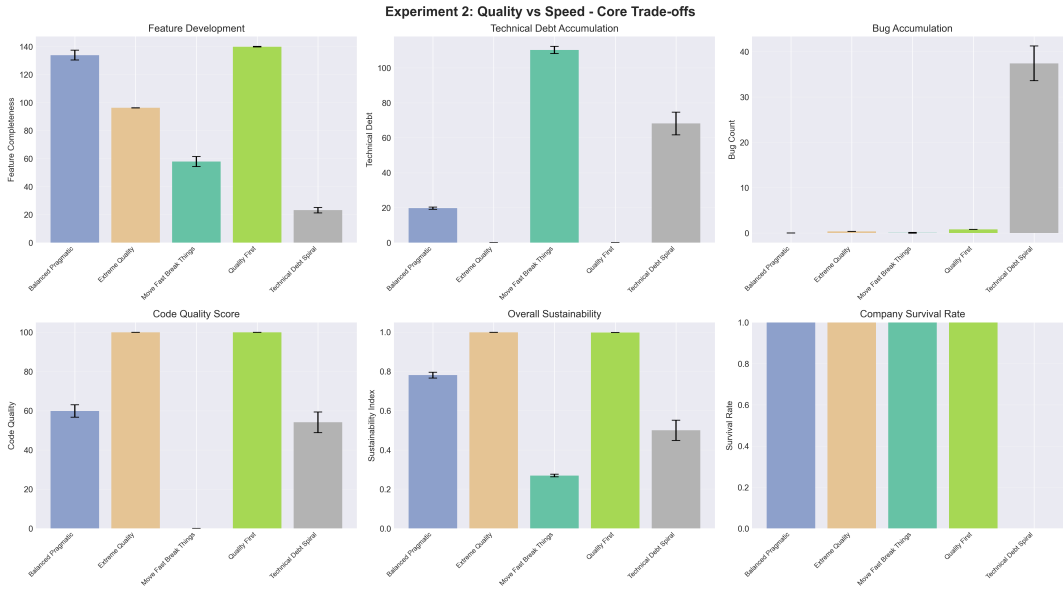


**FIGURE 4** Experiment 2: Core tradeoff analysis. Top: Features, debt, and bugs stratify by strategy. Bottom: Code quality, sustainability, and survival bifurcate—quality strategies maintain constant productivity while debt strategies decay exponentially.

Three mechanisms explain quality feedback activation: exponential debt drag imposes 96–99.6% velocity penalties at debt=68–110; bug cascade thresholds trigger at bugs>20; and diminishing refactoring returns prevent debt recovery when generation exceeds capacity. Sustained velocity beats initial velocity (Quality First delivers 2.4× more features over 200 months), and revenue correlates with sustainability, not velocity.

**TABLE 6** Experiment 2: Performance and quality metrics by strategy (mean ± std)

| Strategy | Surv. | Features | Debt | Bugs | Revenue | Sustain. |
|---|---|---|---|---|---|---|
| Quality First | 100% | 139.89 | 0.00 | 0.80 | $142.20 | 0.998 |
| Extreme Quality | 100% | 96.24 | 0.00 | 0.34 | $126.69 | 0.999 |
| Balanced Pragmatic | 100% | 133.88 | 19.70 | 0.00 | $118.80 | 0.780 |
| Move Fast | 100% | 57.91 | 110.20 | 0.03 | $41.07 | 0.269 |
| Debt Spiral | 0% | 23.16 | 68.14 | 37.42 | $4.00 | 0.500 |

## 4.3 | Experiment 3: Initial Capital and Financial Constraints

Experiment 3 tests seven capital levels from 30 months (bootstrapped) through 300 months (well-funded) across 140 simulation runs with identical teams (10 engineers, 5 sales, 3 marketing, 1 HR, 1 management), isolating financial constraint effects on survival.

Results reveal a sharp survival threshold at 200 months: configurations below experience 100% mortality (120/120 failures), while Series B (200 months) achieves 55% survival and Well Funded (300 months) achieves 100% (Table 7). Increasing capital from 150 to 200 months shifts survival from 0% to 55%, demonstrating financial constraints as hard survival determinants.

**TABLE 7** Experiment 3: Survival outcomes by initial capital level

| Capital Level | Initial Runway | Survival Rate | Mean Survival Time | Survivors |
|---|---|---|---|---|
| *Below Survival Threshold (100% Mortality)* | | | | |
| Bootstrapped | 30 months | 0% | 3.7 ± 0.57 months | 0/20 |
| Seed Stage | 50 months | 0% | 5.9 ± 0.72 months | 0/20 |
| Series A Low | 75 months | 0% | 8.1 ± 0.91 months | 0/20 |
| Series A Standard | 100 months | 0% | 10.2 ± 1.11 months | 0/20 |
| Series A High | 150 months | 0% | 16.05 ± 1.85 months | 0/20 |
| *At or Above Survival Threshold* | | | | |
| Series B | 200 months | 55% | 120.2 ± 90.54 months | 11/20 |
| Well Funded | 300 months | 100% | 200 months (all) | 20/20 |

All 140 runs achieved product-market fit within 15.29 ± 1.36 months regardless of capital level (100% PMF rate). Performance metrics prove capital-insensitive: market share varies 8% (6.87–7.44%), revenue varies 9% ($111–$121), technical debt remains uniform (21.32 ± 0.58). Organizations surviving to PMF converge to identical outcomes regardless of initial funding (Figure 5).

Capital efficiency degrades exponentially: Bootstrapped achieves 0.248 market share per capital month, Well Funded achieves 0.000 (Figure 6). Failure mechanism analysis reveals a 5–7 month profitability gap between PMF achievement (month 15) and cash-flow positivity. Undercapitalized organizations exhaust cash during this gap with linear correlation to initial capital ($r = 0.997$).

These results carry several strategic implications. First, capital functions as a binary survival gate: below the critical threshold, failure is certain regardless of other factors, while above it, performance converges regardless of

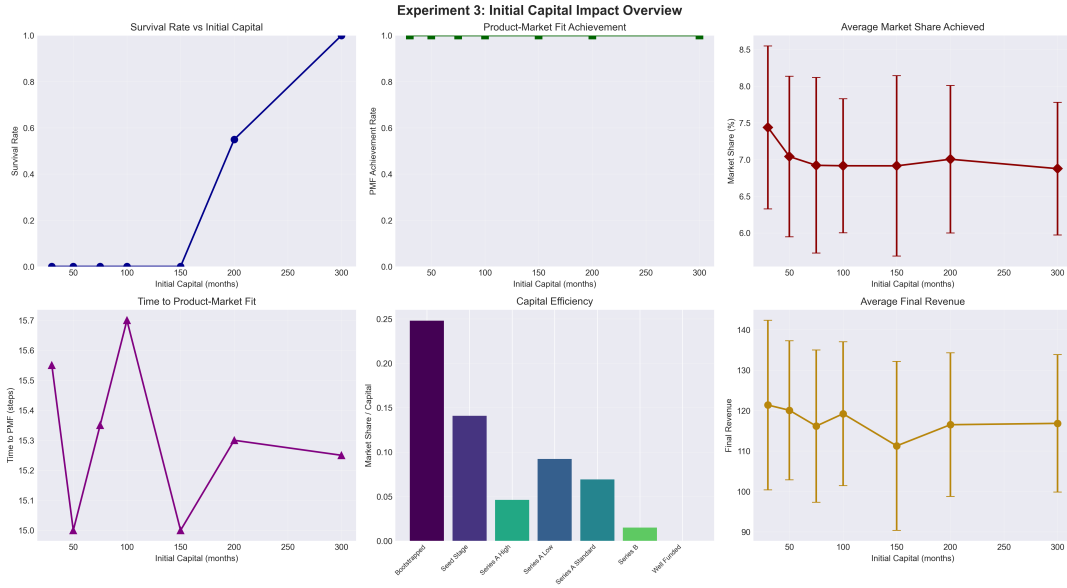


**FIGURE 5** Experiment 3: Initial capital impact overview. Survival rate exhibits sharp threshold at 200 months. PMF achievement uniformly 100%. Market share (6.9–7.4%) and revenue ($111–121) are independent of capital. Capital efficiency decreases exponentially with funding abundance.

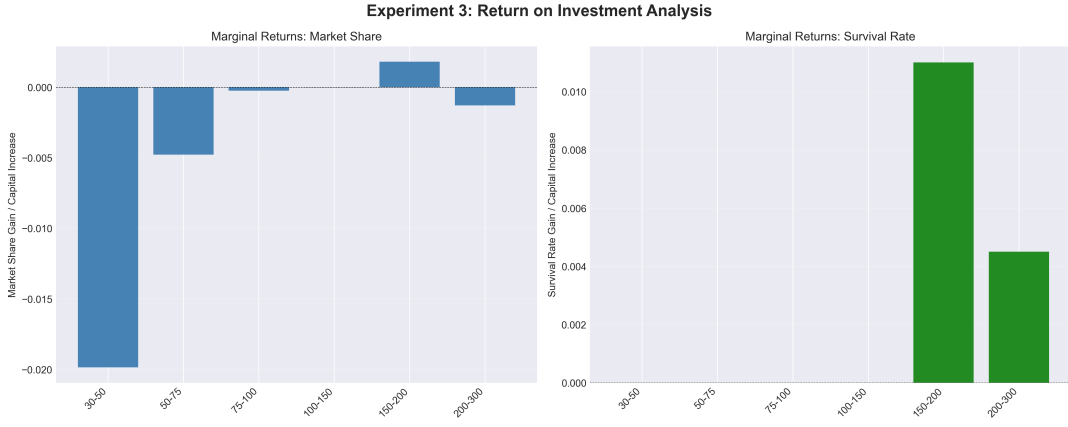


**FIGURE 6** Experiment 3: Return on investment analysis. Left: Marginal returns negative above 150 months. Right: Survival step function at 200-month threshold.

funding abundance. Second, PMF achievement is independent of capital level, suggesting that product development dynamics are governed by team composition rather than financial slack. Third, performance metrics remain largely uniform across a $10\times$ capital range among survivors, indicating that optimal capitalization should target the minimum runway sufficient to bridge the profitability gap rather than maximize funding. The specific 200-month threshold is an artifact of the model parameterization (Section 5) and should be interpreted qualitatively.

## 5 | CONCLUSION

This research developed an agent-based model of software engineering organizations as complex adaptive systems and conducted theoretical exploration through 240 simulation runs across three experiments. Four primary contributions emerge: (1) identification of critical resource thresholds functioning as binary survival gates — capital sufficient to bridge profitability gaps, and dual commercial capacity thresholds ($\geq$6 staff at $\geq$30% composition); (2) demonstration that extreme velocity strategies trigger catastrophic collapse while moderate approaches avoid debt spirals; (3) demonstration that resource thresholds dominate strategic optimization — capital abundance yields no performance improvement among survivors; and (4) methodological demonstration revealing emergent phenomena potentially obscured by survivorship bias. Reported numeric cutoffs are model-dependent and should be read as qualitative indicators of threshold structure.

For practitioners, resource adequacy precedes strategic optimization. Quality-focused approaches paradoxically deliver superior long-term velocity through sustained productivity. Growth-stage companies should recognize exponential capital efficiency degradation with funding abundance. Theoretically, selective activation of quality degradation mechanisms illustrates systems thinking principles — organizations maintaining moderate quality parameters operate in stable equilibria, while extreme strategies cross threshold boundaries entering unstable regimes.

Limitations include model simplifications and parameter calibration from literature estimates. Several outcomes exhibit near-perfect convergence (alignment $\approx$ 100%, conflict $\approx$ 0, PMF 100%) reflecting baseline parameterization with strong stabilizing feedbacks. The 200-month capital threshold is unrealistic for startups, reflecting conservative revenue scaling — recalibration would likely yield 24–36 months. Uniform talent distributions [30, 100] abstract from extreme talent heterogeneity; testing heavy-tailed distributions is a natural extension. Future work should validate revenue parameters, explore talent distribution effects, and incorporate competitive dynamics. The central lesson remains: with inadequate resources, no strategy prevents failure; with adequate resources, diverse strategies may succeed.

## Conflict of interest

The authors declare that they have no conflicts of interest to report regarding the publication of this paper.

## References

Joseph Blackburn, Michael A Lapre, and Luk N Van Wassenhove. Brooks' law revisited: improving software productivity by managing complexity. *Available at SSRN 922768*, 2006.

Barry W. Boehm and Richard Turner. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, Boston, MA, 2003.

David G Chernoguz. The system dynamics of brooks' law in team production. *Simulation*, 87(11):947–975, 2011.

Alexandre Costa, Felipe Ramos, Mirko Perkusich, Emanuel Dantas, Ednaldo Dilorenzo, Ferdinandy Chagas, André Meireles, Danyllo Albuquerque, Luiz Silva, Hyggo Almeida, and Angelo Perkusich. Team formation in software engineering: A systematic mapping study. *IEEE Access*, 8:145687–145712, 2020. doi: 10.1109/ACCESS.2020.3015017.

Richard M Crowder, Mark A Robinson, Helen PN Hughes, and Yee-Wai Sim. The development of an agent-based modeling framework for simulating engineering team work. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, 42(6):1425–1439, 2012.

Xiaocong Fan and John Yen. Modeling and simulating human teamwork behaviors using intelligent agents. *Physics of life reviews*, 1(3):173–201, 2004.

Eduardo Ferreira Franco, Kechi Hirama, Stefano Armenia, and Joaquim Rocha dos Santos. A systems interpretation of the software evolution laws and their impact on technical debt management and software maintainability. *Software Quality Journal*, 31(1):179–209, 2023.

Ramy I Hindiyeh, William K Ocloo, and Jennifer A Cross. Systematic review of research trends in engineering team performance. *Engineering Management Journal*, 35(1):4–28, 2023.

Frederick P. Brooks Jr. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, Reading, MA, anniversary edition edition, 1995.

Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. Technical debt: From metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012.

Samuel Lapp, Kathryn Jablokow, and Christopher McComb. Kaboom: an agent-based model for simulating cognitive style in team problem solving. *Design Science*, 5:e13, 2019.

Bengee Lee and James Miller. Multi-project software engineering analysis using systems thinking. *Software Process: Improvement and Practice*, 9(3):173–214, 2004.

Meir M Lehman. Feedback in the software evolution process. *Information and Software technology*, 38(11):681–686, 1996.

Meir M Lehman and Juan F Ramil. The impact of feedback in the global software process. *Journal of Systems and Software*, 46 (2-3):123–134, 1999.

Raymond J. Madachy. *Software Process Dynamics*. Wiley-IEEE Press, Hoboken, NJ, 2008.

Donella H. Meadows. *Thinking in Systems: A Primer*. Chelsea Green Publishing, White River Junction, VT, 2008.

John Meluso, Jesse Austin-Breneman, and Lynette Shaw. An agent-based model of miscommunication in complex system engineering organizations. *IEEE Systems Journal*, 14(3):3463–3474, 2019.

John Meluso, Jesse Austin-Breneman, James P Bagrow, and Laurent Hébert-Dufresne. A review and framework for modeling complex engineered system development processes. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 52(12): 7679–7691, 2022.

John H. Miller and Scott E. Page. *Complex Adaptive Systems: An Introduction to Computational Models of Social Life*. Princeton University Press, Princeton, NJ, 2007.

Ashkan Negahban and Levent Yilmaz. Agent-based simulation applications in marketing research: an integrated review. *Journal of Simulation*, 8(2):129–142, 2014.

Michael J. North and Charles M. Macal. *Managing Business Complexity: Discovering Strategic Solutions with Agent-Based Modeling and Simulation*. Oxford University Press, New York, NY, 2007.

Doncho Petkov, Denis Edgar-Nevill, Raymond Madachy, and Rory O'Connor. Information systems, software engineering, and systems thinking: Challenges and opportunities. *International Journal of Information Technologies and Systems Approach (IJITSA)*, 1(1):62–78, 2008.

Nelson P. Repenning. Understanding fire fighting in new product development. *Journal of Product Innovation Management*, 18 (5):285–300, 2001.

Daniel Sandria-Flores, Carmen Mezura-Godoy, and Edgard Benítez-Guerrero. Agent-based modeling and simulation for collaborative software: A systematic literature review. In *2024 12th International Conference in Software Engineering Research and Innovation (CONISOFT)*, pages 117–126. IEEE, 2024.

Peter M. Senge. *The Fifth Discipline: The Art and Practice of the Learning Organization*. Doubleday, New York, NY, 1990.

Ian Sommerville. *Software Engineering*. Pearson, Boston, MA, 10th edition, 2016.

John D. Sterman. *Business Dynamics: Systems Thinking and Modeling for a Complex World*. McGraw-Hill, Boston, MA, 2000.

Uri Wilensky and William Rand. *An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo*. MIT Press, Cambridge, MA, 2015.

David C Wynn and P John Clarkson. Process models in design and development. *Research in engineering design*, 29(2):161–202, 2018.

Mohammad Reza Zali, Mina Najafian, and Amir Mohammad Colabi. System dynamics modeling in entrepreneurship research: A review of the literature. *International Journal of Supply and Operations Management*, 1(3):347, 2014.